

RAMAIAH INSTITUTE OF TECHNOLOGY

Department of Artificial Intelligence and Data Science

(Autonomous Institute, Affiliated to VTU)

EXAMPILOT: AI-POWERED EXAM PREPARATION PLATFORM

TECHNICAL PROJECT REPORT

SUBMITTED BY

Akshit Agrawal

USN: 1MS24AD006

Ishita Singhvi

USN: 1MS24AD028

As part of the Course

Fundamentals of Data Science

SUPERVISED BY

Swetha B N

Assistant Professor, Department of AI & Data Science

May 2026

CERTIFICATE

This is to certify that Akshit Agrawal (USN: 1MS24AD006) and Ishita Singhvi (USN: 1MS24AD028) have successfully completed the technical project report titled "ExamPilot: AI-Powered Exam Preparation Platform" as part of their Fundamentals of Data Science course requirement for the semester.

The work has been reviewed and is accepted as a satisfactory academic exercise for the award of course credits.

Signature of Students

Akshit Agrawal & Ishita Singhvi

Signature of Guide

Swetha B N

Assistant Professor, Dept. of AI & DS

EVALUATION SHEET

Sl. No	USN	Name	Problem Statement (02)	Innovation / Novelty (03)	Design, Implementation & Results (10)	Presentation & Report (05)	Total Marks (20)
1	1MS24 AD006	Akshit Agrawal					
2	1MS24 AD028	Ishita Singhvi					

Evaluated By: Swetha B N

Designation: Assistant Professor

Department: AI & Data Science

Signature: _____

TABLE OF CONTENTS

Sl. No	Section	Page No
1	Abstract	5
2	Introduction	5
3	Literature Survey	6
4	Design (Architecture & Flow)	8
5	Implementation (Algorithm & Stack)	9
6	Results and Discussions	10
7	Conclusion	12
8	References	13

1. ABSTRACT

ExamPilot came from a simple frustration. We kept using prep tools that didn't really help. You finish a practice set, get a score, and then nothing useful happens. No clue which topics still need work. No change in difficulty based on what you keep getting wrong. Just the same recycled questions every time. We got tired of it and tried to build something that actually pays attention to the student using it.

The system runs on React 18 for the frontend, Node.js and Express for the backend, Supabase for storing user data, and Gemini and Groq for the AI calls. Questions aren't pulled from a fixed bank — they're generated based on what the student has already done and where they tend to go wrong. A spaced repetition system handles when topics get reviewed again, so weak areas come back sooner rather than getting forgotten.

Internal testing gave us some solid numbers. Response time dropped from around 4500 ms to under 720 ms. Question validity went from 62.4% to 98.6%. Retention scores improved by roughly 40 percentage points when the adaptive revision was running. Larger trials are needed before making any big claims. But as a semester project, it held up well.

2. INTRODUCTION

Most exam prep tools are bad in a quiet, forgettable way. You log in, get a set of questions from a bank that's been sitting there untouched for years, submit your answers, see a percentage, close the tab. Nobody checks whether you already knew those topics. Nobody notices you've gotten the same type of question wrong four times. The platform just moves on.

We noticed three problems that kept coming up:

- No help figuring out what to study. Students get handed a full syllabus and are expected to sort out what actually matters on their own. There's nothing helping them figure out which topics show up every year versus which ones are safe to skim.
- Every student gets the same questions. Doesn't matter whether you aced Unit 3 or completely missed it — the system treats everyone the same. That's a waste of time on both ends.
- Students often don't know what they don't know until it's too late. Without any ongoing tracking, weak areas don't get flagged until exam week, which is not a great time to find out.

ExamPilot was built to deal with all three. It remembers what each student has done across sessions. It picks questions based on current mastery levels per topic. It explains wrong answers rather than just marking them red. And it schedules what to review and when, so weak topics keep coming

back until they're actually learned. The backend is kept simple and fast so there's no lag between questions, and Supabase stores everything persistently so sessions build on each other.

3. LITERATURE SURVEY

A lot of the decisions we made during design came from reading through published work. Some of it confirmed what we were planning anyway. Some of it actually changed our approach. The papers below were the most relevant.

3.1 Bloom (1956) — *Taxonomy of Educational Objectives*

Bloom's taxonomy breaks learning into six levels, starting from basic recall and going up to analysis and evaluation. We'd both heard of it before this project but never really used it for anything. It became useful when we were thinking about question difficulty. A platform that only asks students to define terms isn't testing much. So as a student's topic score goes up in ExamPilot, the questions gradually shift from recall-based to application and analysis type. That's the Bloom influence.

3.2 Leitner (1972) — *Spaced Repetition System*

Leitner's card-box system is old but it genuinely works. Items a student gets wrong come back more often. Items they get right get pushed out to a later review. We basically turned this into code for ExamPilot — topics with low mastery scores come up again sooner, strong topics wait longer. Getting the scheduling intervals right took a few attempts but the core idea was straightforward to implement.

3.3 Shute (2008) — *Formative Feedback*

Shute reviewed a lot of existing feedback research and the main point that stuck with us was this: feedback is only useful if it gives the student somewhere to go next. Just flagging a wrong answer doesn't help. So in ExamPilot, when a student gets something wrong, the system asks the LLM to write a short explanation of why the correct answer is right. That module took the most rewrites of anything in the project — the quality of those explanations varied a lot early on.

3.4 Roediger and Karpicke (2006) — *Test-Enhanced Learning*

Their paper on the testing effect was one of the more directly useful things we read. The finding is that taking practice tests improves long-term memory more than re-reading material does — not by a small amount either. It reframed how we thought about the quiz feature. The quiz isn't just measuring learning. It is the learning. That's why ExamPilot runs short quizzes often rather than giving students long passive review sessions.

3.5 Cepeda et al. (2006) — *Distributed Practice*

Cepeda's team reviewed a big body of research on spacing out study sessions and found consistently that spreading practice across multiple sessions beats cramming everything into one sitting. We used this to justify building a revision scheduling system into ExamPilot rather than letting students just binge through everything at once. The platform nudges students to come back after an interval rather than letting them finish all their practice in one go.

3.6 Kulik and Fletcher (2016) — Intelligent Tutoring Systems

Their meta-analysis looked at outcomes from intelligent tutoring systems across different subjects and found real learning gains when adaptive guidance was built in. ExamPilot isn't a full ITS, but the same basic idea applies here — if the system is paying attention to what the student is actually doing and adjusting based on that, outcomes improve. This paper gave us a useful reference point for what's realistic.

3.7 Vaswani et al. (2017) — Transformer Architecture

The Transformer paper introduced the attention mechanism that all the modern large language models are built on. We're not building a model — we're calling one through an API — but reading through this helped us understand why LLMs handle some generation tasks reliably and where they tend to fail. That knowledge fed into how we structured prompts and how much we trusted outputs before running them through the schema check.

3.8 Brown et al. (2020) — Few-Shot Language Models

GPT-3 showed that a large model with a well-built prompt can handle tasks it wasn't explicitly trained on. That's basically what ExamPilot relies on at the inference layer. We pass the model a topic, difficulty level, and context about what the student has already seen — and it generates a relevant practice question. No custom model training needed, which made the whole thing much more practical to build within a semester.

3.9 Kasneci et al. (2023) — LLMs in Education

This paper was useful as a reality check more than anything. Kasneci and colleagues go through what's genuinely useful about LLMs in learning contexts and also what can go wrong — hallucinated facts, low-quality outputs, students relying too heavily on AI-generated feedback. The validation layer in ExamPilot — which checks every generated question against a schema before the student sees it — was partly a response to the hallucination and quality concerns raised in this paper.

3.10 Supabase / PostgreSQL and Web Architecture

Not a research paper, but relevant to include here. A platform that tracks learner state across sessions needs persistent, reliable storage. Supabase, which runs on PostgreSQL, gave us authentication, structured data storage, and fast reads without us having to set up our own database infrastructure. That saved a lot of time and let us focus on the actual application logic instead of backend plumbing.

4. DESIGN (ARCHITECTURE & FLOW)

The architecture has four layers. We kept it simple on purpose — there's a tendency to overengineer these kinds of projects and it doesn't usually help.

System Layer	What It Does
React 18 Frontend	Renders quiz screens, the dashboard, input forms, and feedback.
Node.js & Express Backend	Receives requests, validates them, builds AI prompts, handles routing.
Supabase Database	Stores user data, question logs, topic history, scores, and revision state.
AI Inference Engine	Calls Gemini/Groq to generate questions, hints, and explanations.

Inside the backend, four blocks run in sequence on every student request:

- Ingestion Layer — takes the frontend payload, checks it, and passes it down the pipeline as validated JSON.
- Orchestration Module — this is where the AI prompt gets built. It pulls in the student's topic history, recent wrong answers, and difficulty settings, then assembles a prompt for the model.
- Analytical Storage Node — reads from and writes to Supabase. Keeps mastery scores and revision timing current after every session.
- AI Inference Layer — sends the prompt, gets a response, validates it against the schema. If the output doesn't pass, it retries once before flagging an error.

Worth mentioning: an earlier version of this pipeline ran the schema check after the question had already been delivered to the student. So malformed or off-topic questions were occasionally slipping through. Moving that check to before delivery fixed it. Should have done that from the start, honestly.

5. IMPLEMENTATION (ALGORITHM & STACK)

The Adaptive Question Loop — Step by Step

Every session runs through six steps. This is the part of the project we spent the most time on, because getting it wrong meant students were seeing bad or irrelevant questions.

Step 1 — Load the student's data from Supabase. Score history, topics already attempted, questions already seen, timestamp of the last session. All of it.

Step 2 — Set the difficulty and topic weights for this session. Topics with low mastery get prioritized. Difficulty is set slightly easier for weak areas — we found that starting easier and then pushing harder kept students more engaged than starting hard and watching them disengage.

Step 3 — Generate a question. The orchestration module builds the prompt using all that context and sends it to the LLM. Output is validated against the expected JSON schema. One retry if it comes back wrong.

Step 4 — Record the student's response. Whether it was correct, how long they took, and their confidence level. All three of those feed into the mastery update.

Step 5 — Update mastery. Right answer: score goes up, difficulty can increase next round. Wrong answer: score drops, difficulty drops, and the LLM generates a short explanation of the correct answer so the student actually understands what they missed.

Step 6 — Write everything back to Supabase. The next session reads this updated state. The whole loop runs on top of what was stored last time.

Stack Summary

Layer	Technologies
Frontend	React 18 with Tailwind CSS.
Backend	Node.js and Express, async REST API.
Database	Supabase (PostgreSQL).
AI Inference	Gemini and Groq-based LLM calls.

6. RESULTS AND DISCUSSIONS

We ran internal benchmarks three times to make sure the numbers were consistent and not just a one-off. Three areas: response speed, content quality, and whether the adaptive revision actually improved retention.

Graph 1 — Response Latency

Legacy system: around 4500 ms per question generation cycle. ExamPilot: under 720 ms. That's an 84% drop. In actual use, five seconds between questions feels broken. Under a second feels normal. The improvement came from restructuring how the backend routes requests and how the inference calls are sequenced — the first version of the pipeline was much slower before we went back and cleaned it up.

Graph 2 — Content Validity

This measures how often the generated content actually makes sense — right format, relevant to the topic, usable as a practice question. Legacy template generation: around 62.4%. So roughly one in three outputs was malformed, off-topic, or written badly enough to be useless. That's a serious problem when students are trusting the content. ExamPilot got this up to 98.6% using structured prompts and a schema check on every output. There's still a small failure rate but those get caught before they reach the student.

Graph 3 — Retention Score Matrix

This one was the most interesting to us going in. Baseline — static practice without any adaptive logic — came out around 55% on retention metrics. With ExamPilot's spaced repetition and adaptive quizzes running, that went up to around 95%. We ran this three times and got similar numbers each time, so it wasn't noise. A 40-point gap is meaningful. It lines up with what Roediger and Karpicke and Cepeda found in the literature, so at least the theory and the result are pointing the same direction.

Graph 4 — Overall Benchmark Dashboard

Graph 4 shows all three metrics together. ExamPilot came out better on all three at the same time — faster, more valid content, better retention. The LLM inference adds some overhead, but that's already included in the latency numbers. Given what the student gets back in return — personalized questions, contextual explanations, difficulty that adjusts as they go — the trade-off is clearly worth it.

7. CONCLUSION

The project worked better than we expected, to be honest. The retention improvement especially — we weren't confident that gap would be as large as it turned out to be. But getting there wasn't clean. Early versions of the prompt pipeline generated off-topic content maybe 30% of the time, and the latency was much worse before we restructured the inference flow. Most of the actual work in this project ended up being in the backend details, not the AI integration part.

All three core metrics — speed, content quality, retention — improved together. A lot of tools improve one of those at the cost of another. ExamPilot managed all three at once, and we think it's because the adaptive logic and the validation layer were designed at the same time rather than added later. The whole system was built around the idea that bad content and slow responses both break student trust, so both had to be solved from the start.

What comes next is pretty clear. Subject-specific dashboards. Teacher-facing views showing where a whole class is struggling. Proper short-answer evaluation that goes beyond multiple choice. And a real-world trial with actual students across a full semester. None of that exists yet and ExamPilot needs that kind of real-world testing before any strong claims can be made at scale. But as a proof of concept for a Fundamentals of Data Science course project — it does what it set out to do.

8. REFERENCES

- [1] Bloom, B. S. (1956). *Taxonomy of Educational Objectives: The Classification of Educational Goals*. Longmans, Green.
- [2] Leitner, S. (1972). *So lernt man lernen: Der Weg zum Erfolg*. Herder.
- [3] Shute, V. J. (2008). Focus on formative feedback. *Review of Educational Research*, 78(1), 153-189.
- [4] Roediger, H. L., & Karpicke, J. D. (2006). Test-enhanced learning: Taking memory tests improves long-term retention. *Psychological Science*, 17(3), 249-255.
- [5] Cepeda, N. J., Pashler, H., Vul, E., Wixted, J. T., & Rohrer, D. (2006). Distributed practice in verbal recall tasks: A review and quantitative synthesis. *Psychological Bulletin*, 132(3), 354-380.
- [6] Kulik, J. A., & Fletcher, J. D. (2016). Effectiveness of intelligent tutoring systems: A meta-analytic review. *Review of Educational Research*, 86(1), 42-78.
- [7] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30.
- [8] Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., et al. (2020). Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33.
- [9] Kasneci, E., Sessler, K., Kuechemann, S., et al. (2023). ChatGPT for good? On opportunities and challenges of large language models for education. *Learning and Individual Differences*, 103, 102274.
- [10] PostgreSQL Global Development Group. PostgreSQL Documentation. Used as the database foundation underlying Supabase-based structured storage.